

Nodejs session 4

▼ Coding

```
// File: index.js
const http = require('http');
const fs = require('fs');
const port = 5000;

const homePage = fs.readFileSync('./views/index.html', 'utf-8');

const server = http.createServer((req, res) => {
  console.log("====Req.Url====");
  console.log(req.url);
  if (req.url === '/') {
    res.write(homePage);
  } else if (req.url === '/about') {
    res.write("<h1>About Page</h1>");
  } else {
    res.write("<h1>Not Found Page</h1>");
  }

  // res.write(JSON.stringify({
  //   id: 1,
  //   name: 'John Doe'
  // }));
  res.end();
});

server.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

```

//File: app.js --> Talking Express
const express = require('express');
const app = express();
const portExpress = 3000;

// app.use(express.static('./views'));
app.use((req, res, next) => {
    console.log(`METHOD: ${req.method} URL: ${req.url}`);
    next();
});

app.get('/', (req, res) => {
    res.send('Welcome From Express!');
});
app.get('/about', (req, res) => {
    res.send('About Page');
});
app.get('/products', (req, res) => {
    res.send([
        { id: 1, name: 'Product 1', price: 10 },
        { id: 2, name: 'Product 2', price: 20 },
        { id: 3, name: 'Product 3', price: 30 }
    ])
});
app.listen(portExpress, () => {
    console.log(`Express server running at http://localhost:${portExpress}/`);
});

```

▼ This Is Explanation

0:00 Recap (مراجعة سريعة)

إنت في السيشنات اللي فاتت اتعلمت:

- إزاي تشغل Node
- إزاي تعمل Server بـ `http`
- إزاي تتعامل مع request / response

عملي

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000);
```

بيحصل إيه؟

• أي request بيجي → يدخل على الـ callback

- `req` = بيانات العميل
- `res` = رد السيرفر

غلطة شائعة

ناس فاكدة إن ده كفاية لبناء API ❌
→ ده low level جداً

Use Case

ده الأساس اللي كل frameworks زي Express مبنية عليه

0:32 More About HTTP Server الفكرة

HTTP Server بيعتمد على:

- URL
- Method (GET / POST)
- Headers
- Body

عملي (Routing يدوي)

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.end("Home");
  } else if (req.url === "/about") {
    res.end("About");
  } else {
    res.end("Not Found");
  }
});

server.listen(3000);
```

اللي بيحصل

- Node بيشفوف `req.url`

- إنت بتعمل routing بإيدك

عملي (Method check) 🔥

```
if (req.method==="POST") {  
  res.end("POST Request");  
}
```

مشاكل الطريقة دي ⚠️

- الكود بيكبر بسرعة 🦴
- صعب تعمل parsing لل body
- مفيش تنظيم

🔥 Use Case (Pentester 🐉)

لو السيرفر معمول بالطريقة دي:

- غالباً فيه bugs
- ممكن تلاقي:
 - Missing validation
 - Improper routing
 - Method confusion

🚀 34:25 Express (الفكرة (النقلة الكبيرة) 🧠

Express = Framework ييسهل:

- Routing
- Middleware
- Request handling

عملي (أول Express Server) 🖥️

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Home");
});

app.get("/about", (req, res) => {
  res.send("About");
});

app.listen(3000);
```

الفرق عن http ⚙️

بدل:

```
if (req.url === "/")
```

بقي:

```
app.get("/")
```

أنصف بكتير 🔥

عملي (POST + JSON) 🔥

```
app.use(express.json());

app.post("/login", (req, res) => {
  console.log(req.body);
  res.send("Done");
});
```

غلطة شائعة ⚠️

نسيان:

```
app.use(express.json());
```

→ `req.body` undefined 🦴

🔥 Use Case

- APIs
- Backend apps
- REST services

🧩 أهم مفهوم في (47:45 Middleware Express)

الفكرة 🧠

Middleware = function route قبل ما يوصل لل request بتمر على كل

عملي (Middleware بسيط) 💻

```
app.use((req,res,next) => {  
  console.log("Request received");  
  next();  
});
```

اللي بيحصل ⚙️

1. request يدخل

2. يمر على middleware

3. `next()` → يروح للي بعده

❌ لو نسيت `next()`

```
app.use((req,res,next) => {  
  console.log("وقف هنا 🤖");  
});
```

→ السيرفر يهجم 😊

🔥 عملي (Auth Middleware)

```
const auth = (req,res,next) => {  
  if (req.headers.token === "123") {  
    next();  
  } else {  
    res.send("Unauthorized");  
  }  
};  
  
app.get("/admin", auth, (req,res) => {  
  res.send("Welcome Admin");  
});
```

🔥 Use Case حقيقي

يستخدم في Middleware:

- Authentication
- Logging
- Rate Limiting
- Security

🐈 ك Pentester ⚠️

دور على:

- Middleware bypass
- Missing auth

• ترتيب middleware غلط

🔥 ربط كل حاجة (Mini Real App)

```
const express = require("express");
const app = express();

app.use(express.json());

// Middleware
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

// Route
app.post("/data", (req, res) => {
  res.json({
    message: "Received",
    data: req.body
  });
});

app.listen(3000);
```

🎯 الخلاصة (مهم جداً)

- `http` → low level
- `Express` → سهل + منظم
- `Middleware` → قلب Express

- `next()` → لازم تفهمها كويس

- أي bug في middleware = ثغرة